Platform Deployment Specification

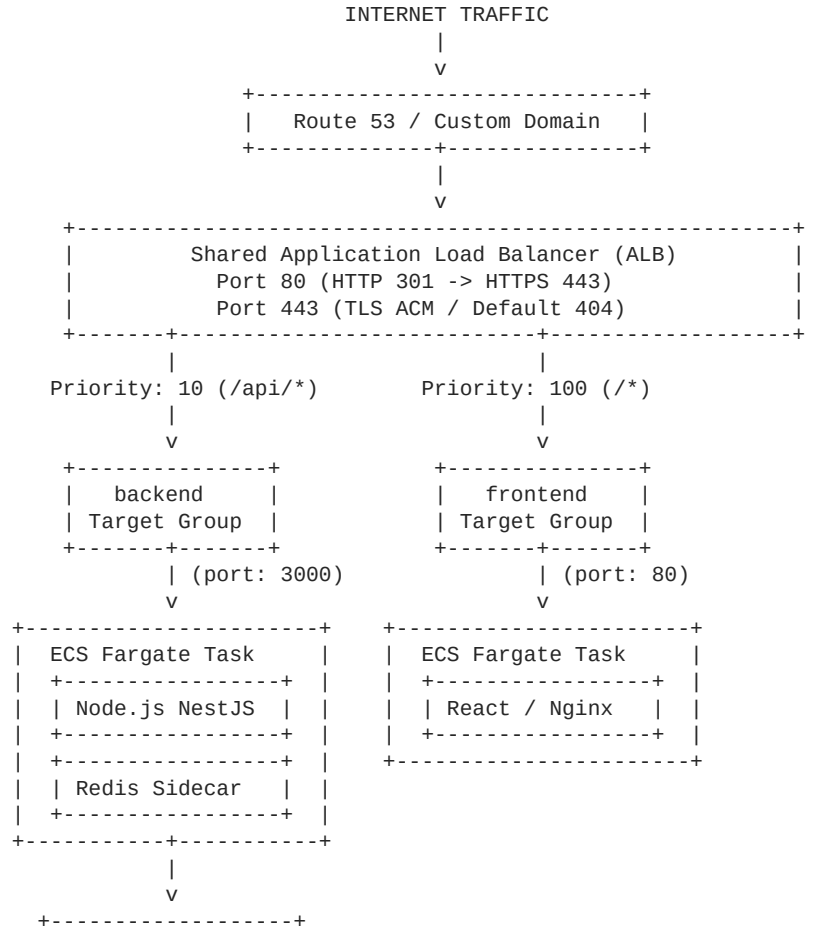
Spec-Driven Development (SDD) & Test-Driven Development (TDD) Guide

01. Architecture Overview

1. Executive Summary

Platform Ops targets containerized microservices on Amazon Web Services (AWS) in a configured region. The system separates the operational topology into two decoupled layers:

1. **The Lifecycle / Platform Foundation (‘<infra-repo>‘):** Shared, persistent infrastructure providing networking, load balancing, DNS, TLS termination, container orchestration clusters, shared execution IAM roles, and self-hosted CI/CD runners.
2. **The Microservice Application Layer (‘backend‘, ‘frontend‘, etc.):** Decoupled repositories hosting application logic, Docker containers, declarative deployment manifests (‘config.yaml‘), and dedicated ECS services.



```
        | MongoDB EC2 / AMI |
        |  (Standalone DB)  |
        +-------------------+
```

---

2. Component Directory

Component	Responsibility	Repository / Module
Shared Network & ALB	Default VPC, public subnets, ingress ALB, ACM TLS cert, port 443 listener rules	‘<env-module>‘
ECS Cluster	‘<ecs-cluster>‘ cluster with ‘FARGATE‘ and ‘FARGATE_SPOT‘ capacity providers	‘<env-module>/ecs.tf‘
Application Module	Standardized, reusable Terraform module generating task definitions, roles, services, and target groups	‘<app-module>‘
CI/CD Runners	Ephemeral GitHub Actions runners on AWS CodeBuild with dedicated VPC access and ECR pull cache	‘<env-module>/codebuild.tf‘
Configuration Store	AWS Systems Manager (SSM) Parameter Store & AWS Secrets Manager	AWS native via IAM scoped roles
Backend App	NestJS REST API with Redis cache sidecar and MongoDB connectivity	‘<backend-repo>/deploy/dev‘
Frontend App	React SPA served via Nginx with client-side routing	‘<frontend-repo>/deploy/dev‘

---

3. Communication & Ingress Patterns

1. **Ingress Routing:**
   * All external HTTP requests arrive at port 80 of the shared ALB and receive an immediate ‘301 Moved Permanently‘ to port 443 HTTPS.
   * Path patterns evaluate against listener rules in order of priority:
     * ‘/api/*‘ (Priority 10) forwards to ‘backend‘ target group.
     * ‘/*‘ (Priority 100) forwards to ‘frontend‘ target group.
     * Unmatched requests return an immediate ‘404 Not Found‘.
2. **Direct Ports:**
   * For direct backend debugging or dedicated socket listeners, an optional direct HTTPS port (e.g. port 3000) can be attached to the ALB forwarding directly to the backend target group.
3. **Internal App-to-App & Sidecars:**
   * In-process sidecars (e.g., Redis container in ‘backend‘) communicate via ‘localhost:<port>‘ within the Fargate task network namespace.

02. Levels of Responsibility: Lifecycle vs. App

In the platform architecture, responsibility is strictly partitioned between two distinct organizational and technical domains: **Lifecycle (Platform Infrastructure)** and **Application (Service Teams)**.

---

1. Responsibility Matrix

Responsibility Domain	Lifecycle (Platform Infra)	Application (Service Team)
VPC & Subnet Topology	**OWNER**	*Consumer*
Shared Application Load Balancer	**OWNER**	*Consumer*
Route 53 & ACM TLS Certificates	**OWNER**	*Consumer*
ECS Cluster & Capacity Providers	**OWNER**	*Consumer*
Shared Task Execution Role (‘<execution-role>‘)	**OWNER**	*Consumer*
CodeBuild GitHub Actions Runners	**OWNER**	*Consumer*
Database Engines (MongoDB AMI/EC2)	**OWNER**	*Consumer*
Reusable App Module (‘ecs-app-module‘)	**OWNER**	*Consumer*
Service Contract (‘config.yaml‘)	*Validator*	**OWNER**
Application Source Code & Unit Tests	*None*	**OWNER**
Dockerfile & Container Packaging	*None*	**OWNER**
App-specific IAM Role (‘*-task‘)	*Template Provider*	**OWNER (Defines needs)**
Application Ingress Path Patterns & Priority	*Arbitrator*	**OWNER (Declares

```
    intent)** |
   | **SSM Parameters & Secrets Names** | *Provisioner* | **OWNER (Declares intent)** |
   | **Application CI/CD Pipeline Execution** | *Platform Provider* | **OWNER** |
```

---

2. Detailed Breakdown: Lifecycle / Platform Responsibilities

When is Lifecycle Responsible?

1. **Initial Platform Bootstrap:**
   * Allocating subnets, configuring internet gateways, NAT routing, and baseline security groups (‘<tasks-security-group>‘, ‘alb-sg‘).
   * Provisioning the shared Application Load Balancer, default HTTP-to-HTTPS redirect listener, and default 404 handler.
   * Acquiring ACM TLS certificates and configuring domain validation.
2. **Cluster & Compute Provisioning:**
   * Managing the ECS Cluster (‘<ecs-cluster>‘).
   * Configuring capacity provider strategies (‘FARGATE‘ vs. ‘FARGATE_SPOT‘ weights).
3. **Execution-Level IAM & Secrets Plumbing:**
   * Creating and updating the execution role (‘<execution-role>‘) required by the AWS ECS Agent to pull private ECR images, create CloudWatch streams, and decrypt SSM parameters with AWS KMS.
   * Creating secret shells in AWS Secrets Manager for CI/CD tokens (‘<github-token-secret>‘, ‘<module-ssh-key-secret>‘).
4. **Shared Pipeline Runners:**
   * Provisioning CodeBuild runner projects with webhook filters listening to ‘WORKFLOW_JOB_QUEUED‘.
   * Building and caching base runner Docker images (e.g., node runner, terraform runner).
5. **Standardized Application Module Maintenance:**
   * Maintaining, testing, and versioning ‘iac/ecs-app-module‘ with complete test coverage (‘app.tftest.hcl‘).

---

3. Detailed Breakdown: Application Team Responsibilities

When is Application Responsible?

1. **Contract Declaration (‘deploy/<env>/config.yaml‘):**
   * Defining the container listening port (‘PORT: 3000‘).
   * Defining the health check endpoint (‘HEALTH: "/api/v1/health"‘).
   * Defining resource sizing (‘CPU: 256‘, ‘MEMORY: 1024‘, ‘DESIRED_COUNT: 1‘).
   * Defining ALB routing path patterns (e.g. ‘path_patterns: ["/api/*"]‘) and listener rule priority.
   * Declaring required sidecar containers (e.g. ‘redis-alpine‘).
2. **Environment Variables & Secrets:**
   * Declaring non-sensitive variables mapped from SSM Parameter Store (‘/backend/*‘).
   * Declaring sensitive secrets injected into container environment from AWS Secrets Manager (‘JWT_SECRET‘, ‘DB_URI‘).
3. **App Task Permissions:**
   * Enabling AWS SES access (‘SES: true‘) if the service sends transactional email.
   * Specifying S3 bucket access (‘S3_BUCKET: "example-service-files"‘) if the service handles file uploads.
4. **Packaging & Delivery:**
   * Maintaining a secure, multi-stage ‘Dockerfile‘.
   * Executing unit, integration, and lint tests in the CI workflow.
   * Pushing built images to ECR tagged with ‘<commit-sha>-<timestamp>‘.
   * Updating the SSM release tag parameter (‘/ci/<service-name>/image_tag‘).
   * Executing ‘terraform apply‘ on the app’s ‘deploy/<env>‘ directory.

03. Deployment Process

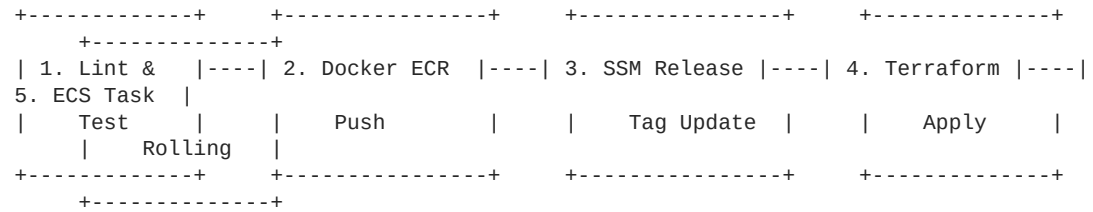
This document details exactly how services and infrastructure changes are deployed across the

platform.

---

1. End-to-End Application Deployment Flow

Every application repository (‘backend‘, ‘frontend‘, or any new service) executes an autonomous, 5-stage deployment pipeline triggered upon push or pull request to the ‘main‘ branch.



Stage 1: Continuous Integration (Lint & Test)

- **Runner:** Ephemeral AWS CodeBuild container (‘codebuild-<runner-
- project>-<run_id>-<run_attempt>‘).
- **Actions:**

* Dependency installation with cache restoration (‘~/.npm‘).
* Static code analysis & linting (‘npm run lint‘).
* Automated unit & integration tests (‘npm run test:ci‘).
* Code coverage report generation.

Stage 2: Container Image Build & Registry Push

- **Actions:**

* Resolve AWS Account ID dynamically via ‘aws sts get-caller-identity‘.
* Compute release tag: ‘IMAGE_TAG="${SHORT_SHA}-${DATE_TAG}"‘ (e.g. ‘41d0162-15-09-26-12-00‘).
* Authenticate Docker daemon against Amazon ECR:
  ‘‘‘bash
  aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin $ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com
  ‘‘‘
* Build container using root ‘Dockerfile‘.
* Push image to ECR repository (‘${ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com/<service-name>:${IMAGE_TAG}‘).

Stage 3: Release Version Registry Update

- **SSM Parameter Store Key:** ‘/ci/<service-name>/image_tag‘
- **Command:**

‘‘‘bash
aws ssm put-parameter --name "/ci/${SERVICE_NAME}/image_tag" --value "${IMAGE_TAG}" --overwrite
‘‘‘

- **Rationale:** Decouples image production from infrastructure updates. Terraform reads
- this parameter via ‘data "aws_ssm_parameter" "image_tag"‘ to determine which container
- image to assign to the ECS task definition.

Stage 4: Terraform Plan & Apply

- **Path:** ‘deploy/<env>/‘
- **Module Resolution:** Configures Git credentials to fetch the private module set by
- ‘APP_MODULE_SOURCE‘ using either the GitHub PAT or Deploy SSH key.
- **Commands:**

‘‘‘bash
terraform init -upgrade
terraform fmt -check
terraform validate

terraform plan
terraform apply -auto-approve
‘‘‘

Stage 5: ECS Fargate Rolling Deployment

- ECS registers a new revision of the task definition with the new image tag.
- Fargate starts new tasks, runs health checks against the target group, verifies healthy
- status, drains existing tasks, and completes zero-downtime rolling update.

---

2. Infrastructure & Lifecycle Deployment Flow

Platform infrastructure changes in ‘<infra-repo>‘ follow a controlled lifecycle:

1. **State Isolation:**
   * Infrastructure state is isolated in S3 (‘<state-bucket>‘, key ‘<state-key>‘).
2. **Execution Steps:**
   ‘‘‘bash
   cd lifecycle/<env>
   terraform init
   terraform plan -out=tfplan
   terraform apply tfplan
   ‘‘‘
3. **Database Changes:**
   * MongoDB AMI changes: Built via HashiCorp Packer (‘mongodb-ami.pkr.hcl‘) and deployed via ‘iac/mongodb/ec2.tf‘.

---

3. Dynamic Configuration & Secrets Provisioning Flow

When an application requires a new environment variable or secret, it is provisioned through the automated workflow in ‘<infra-repo>/.github/workflows/provision-config.yml‘:

1. Developer triggers ‘workflow_dispatch‘ with:
   * Service (‘backend‘, ‘frontend‘, etc.)
   * Type (‘variable‘ or ‘secret‘)
   * Name (‘REDIS_PORT‘ or ‘STRIPE_API_KEY‘)
   * SSM Path (‘/backend/redis_port‘)
   * Secure flag (‘true‘ or ‘false‘)
2. Workflow creates the parameter or secret in AWS with value ‘"PLACEHOLDER"‘.
3. Workflow updates ‘deploy/<service>/<env>/config.yaml‘.
4. Workflow automatically opens a Git branch and Pull Request for review.
5. Admin updates the actual sensitive value in AWS Systems Manager or Secrets Manager before the PR is merged.

04. Client Onboarding Guide & Escalation Protocol

When an engineering team wants to onboard a new application or microservice to the platform, they must complete the **Service Intake Questionnaire**.

The platform provides a set of **predefined shared resources** (in ‘terraform.tfvars‘ and platform modules). If an application’s requirements fit within these defaults, the onboarding is fully automated and self-service. If an application requires custom infrastructure outside these defaults, an **Escalation Request** to the Platform/Lifecycle team is triggered.

---

1. Predefined Platform Infrastructure Values

The following resources are managed centrally by the Platform/Lifecycle team in ‘lifecycle/<env>/terraform.tfvars‘ and ‘iac/ecs-environment-module‘:

```
| Resource | Value to Confirm | Source / Scope |
```

```
| :--- | :--- | :--- |
| **VPC** | Default AWS VPC | Platform Shared |
| **Subnet IDs** | Configured subnet IDs | 'terraform.tfvars:subnet_ids' |
| **ALB Security Group** | '<alb-security-group-id>' | 'terraform.tfvars:alb_sg_id' |
| **Tasks Security Group** | '<tasks-security-group>' |
'data.aws_security_group.tasks' |
| **Shared ALB** | '<project>-dev-alb' | '<env-module>/vpc.tf' |
| **Shared Cluster** | '<ecs-cluster>' (Fargate + Fargate Spot) | '<env-
module>/ecs.tf' |
| **Shared Execution Role**| '<execution-role>' | '<env-module>/rbac.tf' |
| **IAM Role Boundary** | 'arn:aws:iam::<account_id>:policy/example-task-boundary' |
Enforced on all task roles |
| **Central Build ECR** | '<account_id>.dkr.ecr.<region>.amazonaws.com' | ECR repos:
'<builder-ecr-repository>', etc. |
```

---

2. The Intake Questionnaire & Escalation Matrix

Section A: Identity & Ownership

1. **Service Identifier:** Kebab-case service name (e.g. 'cert-service').
2. **Repository Location:** GitHub organization and repo name (e.g. 'example-org/cert-service').
3. **Owning Team & Tech Lead:** Service contact.

Section B: Networking & Ingress (Escalation Check 1)

4. **Standard Question:** What container port does the app listen on? (e.g. '3000', '8080').
5. **Standard Question:** What ALB path pattern should route to this service? (e.g. '["/api/v1/certs/*"]').
6. **Standard Question:** What rule priority should it have? (e.g. '5' for specific routes before '/api/*').
7. ** ESCALATION QUESTION:** *Does your service require a dedicated non-default VPC, private isolated subnets (no public IPs), or custom IP whitelist CIDRs on the ALB?*
   * **If YES -> ESCALATION TO PLATFORM TEAM:**
     * Platform team must provision dedicated subnets or update 'alb_sg_id' ('<alb-security-group-id>') in 'lifecycle/<env>/terraform.tfvars'.
     * The app deployment cannot proceed until the platform team allocates these IDs.
8. ** ESCALATION QUESTION:** *Does your service require a dedicated direct HTTPS port on the ALB (e.g., port 3000 or 8443) or a custom Route53 subdomain (e.g., 'certs.example.com')?*
   * **If YES -> ESCALATION TO PLATFORM TEAM:**
     * Platform team must verify port availability and configure the listener certificate / Route 53 validation records in '<infra-repo>/lifecycle/<env>'.

Section C: Compute & Capacity (Escalation Check 2)

9. **Standard Sizing:** Choose standard Fargate CPU ('256', '512', '1024', '2048') and Memory ('512', '1024', '2048', '4096').
10. ** ESCALATION QUESTION:** *Does your service require dedicated EC2 instances, GPU support, persistent EBS volumes, or compute > 4096 CPU units?*
    * **If YES -> ESCALATION TO PLATFORM TEAM:**
      * Shared cluster '<ecs-cluster>' only provides Fargate and Fargate Spot capacity.
      * Platform team must configure an EC2 or custom capacity provider.

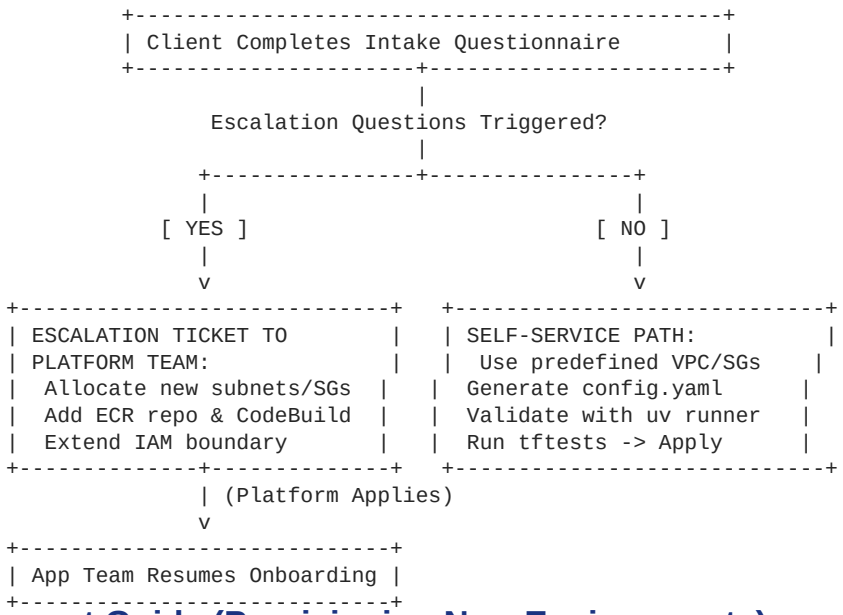
Section D: Cloud Permissions & IAM Boundaries (Escalation Check 3)

11. **Standard Self-Service Permissions:**
    * Amazon SES email sending ('SES: true').
    * Amazon S3 bucket read/write ('S3_BUCKET: "example-service-files"').
    * SSM parameter store reads ('/service/*').
    * Secrets Manager secret reads ('JWT_SECRET', 'DB_URI').

12. ** ESCALATION QUESTION:** *Does your application need access to AWS services outside the standard set (e.g., DynamoDB, SQS, SNS, RDS, KMS, or cross-account roles)?*
    * **If YES -> ESCALATION TO PLATFORM TEAM:**
      * All task roles are gated by the organizational permission boundary ‘example-task-boundary‘.
      * Adding non-standard permissions requires the platform team to extend ‘iac/ecs-app-module/main.tf‘ or provide an approved inline IAM policy template.

---

3. The Onboarding Workflow (Standard vs. Escalated)



05. Environment Guide (Provisioning New Environments)

This guide provides the procedure for adding and configuring a new environment (e.g., ‘stage‘, ‘prod‘, ‘qa‘, or an ephemeral ‘preview‘ environment) to the platform.

---

1. Environment Architecture Principles

Each environment maintains:
1. **Isolated Terraform State:** Stored under ‘<state-bucket>‘ with the configured state key for each deployment. Keep the existing bucket, key, and region when regenerating manifests.
2. **Dedicated Shared ALB:** Following the naming convention ‘<project>-<env>-alb‘ (e.g. ‘example-stage-alb‘, ‘example-prod-alb‘).
3. **Dedicated Subnets & Security Groups:** Or shared VPC with isolated private subnets.
4. **Environment-Scoped Parameters:** Systems Manager parameters partitioned by prefix or account boundary.

---

2. Step-by-Step Environment Provisioning

Step 1: Create the Environment Directory in ‘<infra-repo>‘

Create ‘<infra-repo>/lifecycle/<env>/‘:
- ‘main.tf‘: Instantiates ‘module "platform_ecs"‘ targeting the new environment backend
- state.
- ‘terraform.tfvars‘: Declares subnet IDs, ALB security group, ACM domain, and CodeBuild
- projects for the new environment.
- ‘variables.tf‘: Inherits standard variables.

Step 2: Validate Environment Specification

Run the schema validation against ‘schemas/environment.schema.json‘ and execute the environment contract test:

```
terraform -chdir="/path/to/env-module" test -filter=env_contract.tftest.hcl
```

Step 3: Terraform Apply Infrastructure

Run ‘terraform apply‘ to provision:

- The environment ECS cluster using the configured cluster name.
- The environment ALB with HTTPS listener and ACM certificate.
- The environment CodeBuild runner projects and webhooks.

Step 4: Seed App Configurations

For each active microservice (‘backend‘, ‘frontend‘, etc.):

- Create ‘deploy/<env>/config.yaml‘ populated with environment-specific URLs, database
- names, and routing priorities.
- Create ‘deploy/<env>/main.tf‘, ‘locals.tf‘, ‘providers.tf‘, ‘variables.tf‘, and
- ‘versions.tf‘.
- Populate secrets in AWS Secrets Manager and parameters in AWS SSM.
- Trigger initial deployment.

06. Spec-Driven Development (SDD) Workflow

Spec-Driven Development (SDD) is the operational paradigm used by Platform Ops to ensure all infrastructure, application deployments, and CI/CD pipelines are generated from a single, deterministic contract rather than ad-hoc scripts or prompts.

---

1. Why Spec-Driven Development?

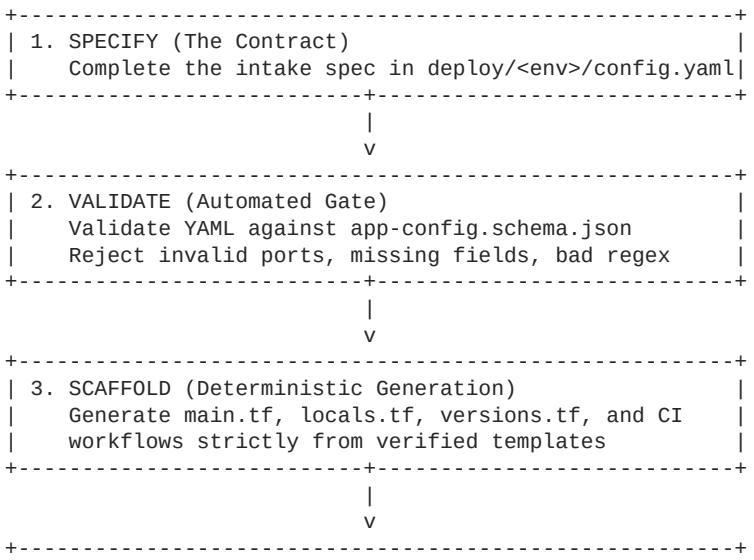
In cloud and AI-assisted development, manual prompting or free-form editing leads to:

- **Configuration Drift:** Missing health checks, incorrect ALB priorities, or improperly
- formatted SSM paths.
- **Security Regressions:** Missing permissions boundaries, insecure environment variables,
- or overly broad IAM policies.
- **Context Hallucinations:** AI assistants guessing non-existent parameters or altering
- critical production defaults.

By using SDD, the **Specification** is the contract. Code generation and deployment tools operate within strict boundaries defined by JSON Schemas.

---

2. The 4-Phase SDD Cycle



```
| 4. APPLY & SYNCHRONIZE                                |
|    Terraform apply and CI execution commit changes.   |
|    If requirements change, update the spec FIRST.     |
+-------------------------------------------------------+
```

---

3. Schema Governance

- The schema definition is stored in ‘schemas/app-config.schema.json‘.
- Validation is executed locally or in CI via:

‘‘‘bash
uv run tests/validate_configs.py <path-to-config.yaml>
‘‘‘

- No Terraform code may be planned or applied if schema validation fails.

07. Test-Driven Development (TDD) for Infrastructure

Test-Driven Development (TDD) for Infrastructure adapts the classic Red-Green-Refactor cycle to Cloud Architecture and Terraform using native test suites.

---

1. The Red-Green-Refactor Loop in Terraform

```
+-------------------------------------------------------+
| 1. RED: Write the Failing Contract Test               |
|    Write assertions in *.tftest.hcl using mock AWS    |
|    providers. Test fails because resource or rule is  |
|    not yet defined.                                   |
+--------------------------+----------------------------+
                           |
                           v
+-------------------------------------------------------+
| 2. GREEN: Implement Minimum IaC / Config              |
|    Update config.yaml or the Terraform module to      |
|    satisfy the assertion. Test passes instantly.      |
+--------------------------+----------------------------+
                           |
                           v
+-------------------------------------------------------+
| 3. REFACTOR: Optimize & Harden                        |
|    Refactor policies, remove redundancies, run fmt &  |
|    validate. All contract tests remain green.         |
+-------------------------------------------------------+
```

---

2. Using the Native Terraform Test Framework

Platform Ops uses Terraform’s native testing framework (‘.tftest.hcl‘), introducing fast, deterministic testing without provisioning real AWS infrastructure:

- **Mock Providers:** ‘mock_provider "aws"‘ mocks all API calls, preventing AWS credential
- dependencies and eliminating cloud costs.
- **Command Mode:** Running with ‘command = apply‘ or ‘command = plan‘ checks HCL logic,
- conditionals, dynamic blocks, and variable validation rules.
- **Instant Feedback:** Tests execute in under 3 seconds.

Running Contract Tests:

```
# Run application contract test suite
terraform -chdir=/path/to/app-module test

# Run environment contract test suite
terraform -chdir=/path/to/env-module test
```

---

3. Test Layers & Gating in CI/CD

1. **Layer 1: Static Analysis** (‘terraform fmt -check‘, ‘terraform validate‘, ‘validate_configs.py‘).
2. **Layer 2: Contract Unit Tests** (‘terraform test‘ with mock providers).
3. **Layer 3: Plan Verification** (‘terraform plan‘).
4. **Layer 4: Deployment Execution** (‘terraform apply -auto-approve‘).

---

4. Security Recommendation: OIDC Federation

> **Note:** The current CI/CD pipeline uses long-lived GitHub PATs for Terraform module access and implicit CodeBuild credentials for AWS API calls. For production hardening, consider migrating to:
> * **GitHub Actions OIDC** with ‘aws-actions/configure-aws-credentials‘ using ‘role-to-assume‘ for short-lived, scoped AWS session tokens.
> * **GitHub App Installation Tokens** instead of PATs for private module access, providing automatic rotation and fine-grained repository permissions.
> * Enforce ‘example-task-boundary‘ on all CI/CD assumed roles, not just ECS task roles.